TEAM NAME

Cypher

Leader: HERWIN

Member 1: SHABAREESH

Member 2: SRI HARI

Table of Contents

Category	Challenge Write-up	Page
REVERSE ENGINEERING	1. Rev	3
REVERSE ENGINEERING	2. Code Maze	4
CRYPTOGRAPHY	3. Caesar Cipher	5
CRYPTOGRAPHY	4. Encoding Chaos	6
CRYPTOGRAPHY	5. Weak Lock	7
FORENSICS	6. Hidden in Plain Sight	8
FORENSICS	7. Pixel Secrets	9
WEB	8. Hidden Profiles	10
WEB	9. Template Terror	11
MISCELLANEOUS	10. Sanity Check (Welcome)	12
MISCELLANEOUS	11. 67	13
MISCELLANEOUS	12. Cipher Master	14

REVERSE ENGINEERING

Rev Write-up

Step 1: Identifying the file

Running file on the challenge binary just said data, and the header was all null bytes. However, running strings showed reversed text like rts_enil_gubed. (.debug_line_str backwards) and fgets@GLIBC_2.2.5 mirrored. The end of the file also had 46 4c 45 7f, which is the ELF magic 7f 45 4c 46 backwards. So the whole file was byte-reversed.

```
d = open("chall", "rb").read()
open("chall_fixed", "wb").write(d[::-1])
```

After reversing the bytes, file reported a normal x86-64 ELF, not stripped, so the function names were still there.

Step 2: Reading the code

I read the binary with objdump and never needed to run it:

- main prints “Back on my feet! What’s the flag?”, reads input with fgets, strips the newline, and calls check_flag.
- check_flag returns 0 unless the input is exactly 24 (0x18) characters long.
- For each byte i, it checks input[i] ^ 0x5a == encrypted_flag[i].
- encrypted_flag sits at 0x402010 in .rodata.

Step 3: Solving

The check is a single-byte XOR, so I XORed the stored bytes with 0x5a to get the flag:

```
enc = bytes.fromhex("150d1b090a213c286a37056d6e6b36056d6a053269 6e3e27".replace("
", ""))
print(bytes(b ^ 0x5a for b in enc).decode())
# OWASP{fr0m_741l_70_h34d}
```

I also checked it against the check_flag logic: the length is 24 and every byte matches.

Flag: `OWASP{fr0m_741l_70_h34d}`

Code Maze Write-up

First look

The challenge gave me one Python file and said it was heavily obfuscated. Opening it, I was surprised by how short it was. There's a big base64 string, and the last line passes it through base64.b64decode, then zlib.decompress, and then exec.

So the real code isn't directly visible in the file. It's packed inside that string and only gets unpacked when the script runs.

What I did

I didn't want to run it blindly, since exec will execute whatever is in there. I swapped exec for print and ran it again:

```
import base64, zlib
data = b"eJxVzLEK..."  # the full string from the file
print(zlib.decompress(base64.b64decode(data)).decode())
```

The hidden code came out like this:

```
import sys
password = input("Enter password: ")
if password == "s3cr3t":
    print("Correct! Flag: OWASP{0bfu5c4t10n_m4k3s_1t_h4rd3r}")
else:
    print("Wrong!")
    sys.exit(1)
```

It's a simple password check. You type s3cr3t and it gives you the flag, otherwise it says "Wrong!" and quits. I could have just run the original script and typed the password, but inspecting the unpacked code first revealed the flag directly.

Flag: `OWASP{0bfu5c4t10n_m4k3s_1t_h4rd3r}`

CRYPTOGRAPHY

Caesar Cipher Write-up

Step 1: Finding the shift

The file says the cipher is Caesar and the shift is somewhere between 1 and 25. I didn't need to brute force it. Every OWASP flag starts with OWASP{, and the ciphertext starts with BJNFC{. Lining them up:

- B → O is +13
- J → W is +13
- N → A is +13 (it wraps around)
- F → S is +13
- C → P is +13

The shift is 13 for every letter, so this is ROT13.

Step 2: Decrypting

Caesar only shifts letters, so the digits and symbols stay the same.

```
def caesar(text, shift):
    out = []
    for ch in text:
        if ch.isalpha():
            base = ord('A') if ch.isupper() else ord('a')
            out.append(chr((ord(ch) - base + shift) % 26 + base))
        else:
            out.append(ch)
    return "".join(out)

ct = "BJNFC{p43f4e_p1cu3e_1f_34fl_g0_pe4px}"
print(caesar(ct, 13))
# OWASP{c43s4r_c1ph3r_1s_34sy_t0_cr4ck}
```

If you didn't know the shift, you could loop shift from 1 to 25 and look for the output that starts with OWASP{.

Flag: `OWASP{c43s4r_c1ph3r_1s_34sy_t0_cr4ck}`

Encoding Chaos Write-up

Step 1: Identifying the encoding

The data uses only letters, digits, +, /, and ends with =. That is the Base64 alphabet. It also starts with Vm, which is what Base64 of Base64 text often looks like (Vm decodes to V, and the output starts with V again).

Step 2: Peeling the layers

I decoded it in a loop until it stopped being valid Base64:

```
import base64

cur = b"Vm10U1IyRXhSbk5X..."   # full string from the file
while True:
    try:
        cur = base64.b64decode(cur, validate=True)
    except Exception:
        break
print(cur.decode())
```

Each layer, shortened:

Layer	Output starts with
1	`VmtSR2ExRnNW...`
2	`VkRGa1FsVXhR...`
3	`VDFkQlUxQjda...`
4	`T1dBU1B7YjRz...`
5	`OWASP{b4s364_1s_n0t_3ncrypt10n}`

The loop stops after layer 5 because the plaintext flag isn't valid Base64.

Flag: `OWASP{b4s364_1s_n0t_3ncrypt10n}`

Weak Lock Write-up

Step 1: Reading the challenge

The file gives 31 bytes of ciphertext in hex and says the cipher is a repeating XOR with a 4-byte key:

```
0e160012113a3672752a1e2a72381e2c752a72321e727532381e3671332a3c
```

Step 2: Known-plaintext attack

XOR is its own inverse, so ciphertext ^ plaintext = key. OWASP flags start with OWASP{, so I XORed the first 6 ciphertext bytes with that prefix:

Cipher	Plain	Key
0e	O (4f)	41
16	W (57)	41
00	A (41)	41
12	S (53)	41
11	P (50)	41
3a	{ (7b)	41

Every byte came out as 0x41, which is the letter A. So the key is AAAA. The 5th and 6th bytes also gave 41, which confirms the key repeats every 4 bytes.

Step 3: Decrypting

```
ct =
bytes.fromhex("0e160012113a3672752a1e2a72381e2c752a72321e727532381e3671332a3c")
key = b"AAAA"
print(bytes(c ^ key[i % 4] for i, c in enumerate(ct)).decode())
# OWASP{w34k_k3y_m4k3s_34sy_w0rk}
```

Flag: `OWASP{w34k_k3y_m4k3s_34sy_w0rk}`

FORENSICS

Pixel Secrets Write-up

Step 1: Checking the easy places first

The file is a 200×150 RGB PNG, only 803 bytes. strings found nothing readable and the PNG had no metadata chunks, so the message wasn't in the file's text. That left the pixel data, and LSB steganography is the usual suspect.

Step 2: Looking at the LSBs

Changing the lowest bit of a color value shifts it by 1 out of 255, which the eye can't see. I took the lowest bit of every R, G, and B value, row by row, and packed the bits into bytes.

- Reading each channel on its own gave junk.
- Reading R, G, B interleaved, pixel by pixel, gave this:

```
00 00 00 26 | OWASP{50_l1ttl3_b1ts_h1d3_b1g_s3cr3t5}
```

The first 4 bytes are a length header: 0x26 = 38. The message after it is exactly 38 characters, which confirms this is the real encoding.

Step 3: Solve script

```
from PIL import Image

im = Image.open("vacation_photo.png").convert("RGB")
bits = [c & 1 for px in im.getdata() for c in px]

raw = bytes(int("".join(map(str, bits[i:i+8])), 2)
            for i in range(0, len(bits) - 7, 8))

length = int.from_bytes(raw[:4], "big")
print(raw[4:4 + length].decode())
# OWASP{50_l1ttl3_b1ts_h1d3_b1g_s3cr3t5}
```

Flag: **OWASP{50_l1ttl3_b1ts_h1d3_b1g_s3cr3t5}**

Hidden in Plain Sight Write-up

Step 1: Reading the dump

Every line has the hex bytes on the left and an ASCII view on the right between the | bars. For most rows, the ASCII side is gibberish like 0.t2..g.....j2.. That is what random or encrypted bytes look like.

Step 2: Spotting the odd rows

Two rows stand out, at offsets 0x80 and 0x90:

```
00000080  4f 57 41 53 50 7b 68 33 78 5f 64 34 6d 70 35 5f  |OWASP{h3x_d4mp5_|
00000090  34 72 33 5f 79 30 75 72 5f 66 72 31 33 6e 64 7d  |4r3_y0ur_fr13nd}|
```

- Their ASCII column is readable text, and it starts with OWASP{.
- Their hex is lowercase, while every other row uses uppercase. That is a sign someone inserted them by hand.

Step 3: Verifying

I decoded the hex myself to check it matches the ASCII column:

Hex	Text
4f 57 41 53 50 7b	OWASP{
68 33 78 5f	h3x_
64 34 6d 70 35 5f	d4mp5_
34 72 33 5f	4r3_
79 30 75 72 5f	y0ur_
66 72 31 33 6e 64 7d	fr13nd}

To find it automatically, run strings on the raw bytes or grep -a "OWASP{".

Flag: `OWASP{h3x_d4mp5_4r3_y0ur_fr13nd}`

WEB

HIDDEN PROFILES Write-up

Step 1: Reading the app

The homepage just says “Visit /profile/1 to see your profile.” That ID sits right in the URL, with nothing checking whether it’s actually your account — a classic Insecure Direct Object Reference.

Step 2: Walking the IDs

I loaded each profile in turn:

URL	User	Role
`/profile/1`	alice	user
`/profile/2`	bob	user
`/profile/3`	charlie	user
`/profile/0`	admin	administrator

Nothing in the app told me to try 0 — it’s just the natural “ID before the first user,” since admin accounts are often seeded first with ID 0 or 1 and regular users start after.

Step 3: The result

/profile/0 returns the admin’s profile with the flag embedded directly in the page:

```
Welcome, admin!
Role: administrator
Email: admin@company.com
Flag: OWASP{1d0r_all0ws_un4uth0r1z3d_4cc3ss_e56ef0c519}
```

No authentication, no session check — just changing the number in the URL was enough.

Flag: `OWASP{1d0r_all0ws_un4uth0r1z3d_4cc3ss_e56ef0c519}`

WEB

Template Terror Write-up

Step 1: Taking a Look

The homepage just renders our input using a template engine

Step 2: Vulnerability detection

To confirm if the server is interpreting our input as code rather than just text, we perform a simple math test. We input **{{ 7 * 7 }}** , The server returns 49. This confirms the application evaluates the code inside the braces. We have a valid injection point.

Step 3: Finding the file name of the flag

Before we can read the flag, we need to know its filename. For that we input the payload: **{{ url_for.__globals__.os.popen('ls').read() }}.** The server returns the file name as app.py.

Step 4: Capturing the flag

Now that we have the correct filename, we modify our payload to read the content of the app.py file using the cat command. **{{ url_for.__globals__.os.popen('cat app.py').read() }}**
This reveals the flag hidden in the app.py file.

Flag: `OWASP{sst1_4ll0ws_rc3_1f_n0t_s4n1t1z3d_a2fdada9ad}`

MISCELLANEOUS

Sanity Check (Welcome)

Challenge

Read the attached welcome.txt file to verify the setup and get on the scoreboard.

Walkthrough

This first challenge for the OWASP SREC CTF was a starter check to make sure everyone knew how to submit flags and get on the scoreboard. The challenge provided a text file named welcome.txt to inspect.

Opening welcome.txt revealed a short welcome message from the student chapter along with the basic competition ground rules. Right at the bottom of the file, the sanity check flag was included in plain text.

Flag: `OWASP{w3lc0m3_t0_0w4sp_sr3c_ctf_2026}`

67

Overview

This is a classic substitution cipher challenge. The entire chall (1).txt file is a large block of repeating 6s and 7s. Anytime a text block is made up of exactly two repeating characters, it points directly to binary — 0s and 1s in disguise.

Step 1: Find the mapping

Since we need 0s and 1s, I mapped 6 = 0 and 7 = 1.

Step 2: Test the first byte

Binary is read in 8-bit chunks (bytes). Taking the first 8 characters from the file (67667777) and applying the mapping gives 01001111. Converting that binary byte into an ASCII character produces the letter 'O'.

Step 3: Confirm the flag format

To confirm the mapping, I checked the next few 8-character chunks:

- 67676777 → 01010111 = W
- 67666667 → 01000001 = A
- 67676677 → 01010011 = S
- 67676666 → 01010000 = P

It spells out OWASP, matching the standard flag format. Instead of decoding the remaining 300+ characters by hand, I wrote a quick Python script to decode the full file:

```
# The text from chall (1).txt
cipher = "676677776767677767666667676766776767666667777677..."

# Swap the 6s and 7s for 0s and 1s
binary_str = cipher.replace('6', '0').replace('7', '1')

# Chunk the string into 8-bit blocks and convert each to an ASCII character
flag = "".join(chr(int(binary_str[i:i+8], 2)) for i in range(0, len(binary_str),
8))

print(flag)
```

Flag: `OWASP{c0mput3r5_d0n7_5p34k_b1n4ry_7h3y_5p34k_67}`

Cipher Master

Overview

The prompt gave a string of encoded data and specified the exact layers of encoding that were applied to it: Base64, ROT13, Base64 again, and finally a string reversal. Since encoding is a fixed operation, solving this just requires running the operations in the exact reverse order.

Unwinding the layers

- **1. Reverse the string:** The encoded data started with =, which is a giveaway for Base64 padding that got flipped around. Reversing the string puts the = back at the end where it belongs.
- **2. Base64 Decode:** Decoding the reversed string produces a new string that looks like gibberish, but is actually shifted text.
- **3. ROT13 Decode:** Applying ROT13 (which is its own inverse) shifts the alphabet back to reveal the first Base64 layer.
- **4. Base64 Decode:** One final Base64 decode on the ROT13 output reveals the plaintext flag.

Solve script

Rather than decoding four layers manually, I wrote a Python script to automate the process:

```
import base64
import codecs

# The raw encoded data from the challenge
encoded_str = "=0TPENXbXNjW1UUUvNXc692aERlW3VjMaNXQR9maTFVcrh0SvdzTxg0TxFzR"

# Step 1: Reverse
step1 = encoded_str[::-1]

# Step 2: First Base64 Decode
step2 = base64.b64decode(step1).decode('utf-8')

# Step 3: ROT13 Decode
step3 = codecs.decode(step2, 'rot_13')

# Step 4: Final Base64 Decode
flag = base64.b64decode(step3).decode('utf-8')

print(flag)
```

Flag: `OWASP{mu1t1pl3_3nc0d1ng_l0c3rs}`